Com S // Cpr E // Math 4240

Introduction to High-Performance Computing

Instructor: Songting Luo

Lecture 15: More on Finding Eigenvalues, and More on Code Optimization

Outline

1. More on Finding Eigenvalues

2. More on code optimization

More on Finding Eigenvalues

Tridiagonal matrix

In linear algebra, a tridiagonal matrix is a matrix that has nonzero elements only on the main diagonal, the first diagonal below this, and the first diagonal above the main diagonal.

$$A = \begin{bmatrix} a_1 & b_1 & & & & & \\ c_2 & a_2 & b_2 & & & & \\ & c_3 & a_3 & b_3 & & & \\ & & c_4 & a_4 & b_4 & & \\ & & & \ddots & \ddots & \ddots & \\ & & & & c_{N-1} & a_{N-1} & b_{N-1} \\ & & & & & c_N & a_N \end{bmatrix}$$

Tridiagonal matrix

$ISUHPC/lectures/lecture15/codes/QRalgorithm/trimatrix.h:

```
1  #ifndef __TRIMATRIX_H__
2  #define __TRIMATRIX_H__
3  #include "matrix.h"
4  typedef struct trimatrix trimatrix;
5  struct trimatrix
6  {
7     int rows; int cols;
8     double* left;
9     double* center;
10    double* right;
11 };
12 typedef struct pentamatrix pentamatrix;
13 struct pentamatrix
14 {
15    int rows; int cols;
16    double* leftleft;
17    double* left;
18    double* center;
19    double* right;
20    double* rightright;
21 };
```

Tridiagonal matrix

$ISUHPC/lectures/lecture15/codes/QRalgorithm/trimatrix.c:

```
1  #include "trimatrix.h"
2
3  trimatrix new_trimatrix(const int rows)
4  {
5     trimatrix trimat;
6     assert(rows>0);
7     trimat.rows   = rows;
8     trimat.cols   = rows;
9     trimat.left   = (double*)malloc(sizeof(double)*rows);
10    trimat.center = (double*)malloc(sizeof(double)*rows);
11    trimat.right  = (double*)malloc(sizeof(double)*rows);
12
13    for (int i=0; i<(rows); i++)
14    {
15       trimat.left[i]   = 0.0;
16       trimat.center[i] = 0.0;
17       trimat.right[i]  = 0.0;
18    }
19
20    return trimat;
21 }
```

Tridiagonal matrix

$ISUHPC/lectures/lecture15/codes/QRalgorithm/trimatrix.c:

```
1  double tget(const trimatrix* trimat, const int i, const int j)
2  {
3     if (i==j)
4     {  return trimat->center[i-1];  }
5     else if (i==(j+1))
6     {  return trimat->left[i-1];    }
7     else if (i==(j-1))
8     {  return trimat->right[i-1];   }
9     else
10    {  return 0.0; }
11 }
12
13 void tset(trimatrix* trimat, const int i, const int j,
14           const double val)
15 {
16    if (i==j)
17    {  trimat->center[i-1] = val;  }
18    else if (i==(j+1))
19    {  trimat->left[i-1]   = val;  }
20    else if (i==(j-1))
21    {  trimat->right[i-1]  = val;  }
22 }
```

Tridiagonal matrix

$ISUHPC/lectures/lecture15/codes/QRalgorithm/trimatrix.c:

```
1  trimatrix grab_sub_trimatrix(trimatrix* T, int k1, int k2)
2  {
3     const int size = k2-k1+1;
4     assert(T->rows>0); assert(k1>0);
5     assert(k2>=k1); assert(k2<=T->rows);
6
7     trimatrix Tnew = new_trimatrix(size);
8     const int shift = k1-1;
9
10    tset(&Tnew,1,1, tget(T,1+shift,1+shift));
11    tset(&Tnew,1,2, tget(T,1+shift,2+shift));
12    for (int i=2; i<=(size-1); i++)
13    {
14       tset(&Tnew,i,i-1, tget(T,i+shift,i+shift-1));
15       tset(&Tnew,i,i,   tget(T,i+shift,i+shift));
16       tset(&Tnew,i,i+1, tget(T,i+shift,i+shift+1));
17    }
18    tset(&Tnew,size,size-1, tget(T,size+shift,size+shift-1));
19    tset(&Tnew,size,size,   tget(T,size+shift,size+shift));
20
21    return Tnew;
22 }
```

Tridiagonal matrix

$ISUHPC/lectures/lecture15/codes/QRalgorithm/trimatrix.c:

```
1  trimatrix merge_sub_trimatrix(trimatrix* T1, trimatrix* T2)
2  {
3     const int k1 = T1->rows; assert(k1>0);
4     const int k2 = T2->rows; assert(k2>0);
5     const int N = k1+k2;
6
7     trimatrix T = new_trimatrix(N);
8
9     tset(&T,1,1, tget(T1,1,1) );
10    tset(&T,1,2, tget(T1,1,2) );
11    for (int i=2; i<=(k1-1); i++)
12    {
13       tset(&T,i,i-1, tget(T1,i,i-1) );
14       tset(&T,i,i,   tget(T1,i,i) );
15       tset(&T,i,i+1, tget(T1,i,i+1) );
16    }
17    tset(&T,k1,k1-1, tget(T1,k1,k1-1) );
18    tset(&T,k1,k1,   tget(T1,k1,k1) );
19
20    tset(&T,k1+1,k1+1, tget(T2,1,1) );
21    tset(&T,k1+1,k1+2, tget(T2,1,2) );
```

Tridiagonal matrix

$ISUHPC/lectures/lecture15/codes/QRalgorithm/trimatrix.c:

```
    for (int i=2; i<=(k2-1); i++)
    {
       tset(&T,k1+i,k1+i-1, tget(T2,i,i-1) );
       tset(&T,k1+i,k1+i,   tget(T2,i,i) );
       tset(&T,k1+i,k1+i+1, tget(T2,i,i+1) );
    }
    tset(&T,k1+k2,k1+k2-1, tget(T2,k2,k2-1) );
    tset(&T,k1+k2,k1+k2,   tget(T2,k2,k2) );

    return T;
}
```

Recall: QR Factorization

Orthogonal Matrices

The matrix $Q \in \mathbb{R}^{N \times N}$ is **orthogonal** if the inverse of Q is equal to its transpose:

$$Q^T = Q^{-1}.$$

Example: rotation matrix

$$Q = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}, \quad Q^T = \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{bmatrix}, \quad Q^T Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

Recall: QR Factorization

Orthogonal Matrices

The matrix $Q \in \mathbb{R}^{N \times N}$ is **orthogonal** if the inverse of Q is equal to its transpose:

$$Q^T = Q^{-1}.$$

Example: rotation matrix

$$Q = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}, \quad Q^T = \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{bmatrix}, \quad Q^T Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

QR Factorization

The matrix $A \in \mathbb{R}^{N \times N}$ can be written as the product of an orthogonal matrix Q and an upper triangular matrix R:

$$A = QR.$$

This factorization is accomplished through the Gram-Schmidt process applied to the columns of A. If A is invertible, then so is R. Q is always invertible.

Finding eigenvalues: Phase 1

Let $A \in \mathbb{R}^{N \times N}$ be a **real** and **symmetric** matrix.

Finding eigenvalues: Phase 1

Let $A \in \mathbb{R}^{N \times N}$ be a **real** and **symmetric** matrix.

Idea #1: It would be nice if we could **easily** find an orthogonal matrix Q such that

$$Q^T A Q = \Lambda \qquad \Longleftrightarrow \qquad A = Q \Lambda Q^T,$$

where Λ is the diagonal matrix of eigenvalues.

Finding eigenvalues: Phase 1

Let $A \in \mathbb{R}^{N \times N}$ be a **real** and **symmetric** matrix.

Idea #1: It would be nice if we could **easily** find an orthogonal matrix Q such that

$$Q^T A Q = \Lambda \qquad \Longleftrightarrow \qquad A = Q \Lambda Q^T,$$

where Λ is the diagonal matrix of eigenvalues.

We know that such a Q exists, because the eigenvectors of a real symmetric matrix are mutually orthogonal.

Finding eigenvalues: Phase 1

Let $A \in \mathbb{R}^{N \times N}$ be a **real** and **symmetric** matrix.

Idea #1: It would be nice if we could **easily** find an orthogonal matrix Q such that

$$Q^T A Q = \Lambda \qquad \Longleftrightarrow \qquad A = Q \Lambda Q^T,$$

where Λ is the diagonal matrix of eigenvalues.

We know that such a Q exists, because the eigenvectors of a real symmetric matrix are mutually orthogonal.

However, finding Q is **hard**. i.e., There is no simple algorithm allows us to build Q step-by-step.

Finding eigenvalues: Phase 1

Let $A \in \mathbb{R}^{N \times N}$ be a **real** and **symmetric** matrix.

Idea #1: It would be nice if we could **easily** find an orthogonal matrix Q such that

$$Q^T A Q = \Lambda \qquad \Longleftrightarrow \qquad A = Q \Lambda Q^T,$$

where Λ is the diagonal matrix of eigenvalues.

We know that such a Q exists, because the eigenvectors of a real symmetric matrix are mutually orthogonal.

However, finding Q is **hard**. i.e., There is no simple algorithm allows us to build Q step-by-step.

Idea #2: What if we could find a finite sequence of (easily constructed) orthogonal matrices, Q_i, such that

$$Q_{N-2}^T \cdots Q_3^T Q_2^T Q_1^T \, A \, Q_1 Q_2 Q_3 \cdots Q_{N-2} = T$$

where $T \in \mathbb{R}^{N \times N}$ is a **real, symmetric, and tridiagonal** matrix.

Finding eigenvalues: Phase 1

Let $A \in \mathbb{R}^{N \times N}$ be a **real** and **symmetric** matrix.

Idea #1: It would be nice if we could **easily** find an orthogonal matrix Q such that

$$Q^T A Q = \Lambda \qquad \Longleftrightarrow \qquad A = Q \Lambda Q^T,$$

where Λ is the diagonal matrix of eigenvalues.

We know that such a Q exists, because the eigenvectors of a real symmetric matrix are mutually orthogonal.

However, finding Q is **hard**. i.e., There is no simple algorithm allows us to build Q step-by-step.

Idea #2: What if we could find a finite sequence of (easily constructed) orthogonal matrices, Q_i, such that

$$Q_{N-2}^T \cdots Q_3^T Q_2^T Q_1^T \, A \, Q_1 Q_2 Q_3 \cdots Q_{N-2} = T$$

where $T \in \mathbb{R}^{N \times N}$ is a **real, symmetric, and tridiagonal** matrix.

It turns out you can always do this in a **finite** number of steps, in fact $\mathcal{O}(N)$ steps. Therefore, total cost is $\mathcal{O}(N^3)$ (same scaling as Gaussian elimination).

Phase 1: Reduction to a tridiagonal matrix

$ISUHPC/lectures/lecture15/codes/QRalgorithm/Hessenberg.c:

```
1  void Hessenberg(const matrix* Ain, trimatrix* T)
2  {
3     const int N = Ain->rows; assert(N>1);
4     assert(N==Ain->cols);
5     assert(N==T->rows); assert(N==T->cols);
6
7     matrix A = new_matrix(N,N);
8     for (int i=1; i<=N; i++)
9        for (int j=1; j<=N; j++)
10       { mget(A,i,j) = mgetp(Ain,i,j); }
11
12    vector x = new_vector(N);
13    vector v = new_vector(N);
14    vector vTH1 = new_vector(N);
15    vector Hv2 = new_vector(N);
16
17    // Main loop
18    for (int k=1; k<=(N-2); k++)
19    {
```

Phase 1: Reduction to a tridiagonal matrix

$ISUHPC/lectures/lecture15/codes/QRalgorithm/Hessenberg.c:

```
        // Part 1: Construct reflector vector "v"
        for (int i=(k+1); i<=N; i++)
        { vget(x,i-k) = mget(A,i,k); }

        double normX = 0.0;
        for (int j=1; j<=(N-k); j++)
        { normX += pow(vget(x,j),2); }
        normX = sqrt(normX);

        for (int j=1; j<=(N-k); j++)
        { vget(v,j) = vget(x,j); }
        vget(v,1) = vget(v,1) + sign(vget(x,1))*normX;

        double normV = sqrt(pow(normX,2) - pow(vget(x,1),2)
                            + pow(vget(v,1),2));

        for (int j=1; j<=(N-k); j++)
        { vget(v,j) = vget(v,j)/normV; }
```

Phase 1: Reduction to a tridiagonal matrix

$ISUHPC/lectures/lecture15/codes/QRalgorithm/Hessenberg.c:

```
        // Part 2: apply reflector to (k+1):N by k:N block of A
        for (int i=1; i<=(N+1-k); i++)
        {
           vget(vTH1,i) = 0.0;
           for (int j=(k+1); j<=N; j++)
           { vget(vTH1,i) += vget(v,j-k)*mget(A,j,i-1+k); }
        }

        for (int i=1; i<=(N-k); i++)
           for (int j=1; j<=(N+1-k); j++)
           {
              int ishift = k+i;
              int jshift = (k-1)+j;
              mget(A,ishift,jshift) -=
                 2.0*vget(v,i)*vget(vTH1,j);
           }
```

Phase 1: Reduction to a tridiagonal matrix

$ISUHPC/lectures/lecture15/codes/QRalgorithm/Hessenberg.c:

```
        // Part 3: apply reflector to 1:N by (k+1):N block of A
        for (int i=1; i<=N; i++)
        {
           vget(Hv2,i) = 0.0;
           for (int j=1; j<=(N-k); j++)
           { vget(Hv2,i) += mget(A,i,j+k)*vget(v,j); }
        }

        for (int i=1; i<=N; i++)
           for (int j=1; j<=(N-k); j++)
           {
              int ishift = i;
              int jshift = j+k;
              mget(A,ishift,jshift) -=
                 2.0*vget(Hv2,i)*vget(v,j);
           }
```

Phase 1: Reduction to a tridiagonal matrix

$ISUHPC/lectures/lecture15/codes/QRalgorithm/Hessenberg.c:

```
    }

    tset(T,1,1,mget(A,1,1));
    tset(T,1,2,mget(A,1,2));
    for (int i=2; i<=(N-1); i++)
    {
       tset(T,i,i-1,mget(A,i,i-1));
       tset(T,i,i,  mget(A,i,i));
       tset(T,i,i+1,mget(A,i,i+1));
    }
    tset(T,N,N-1,mget(A,N,N-1));
    tset(T,N,N,mget(A,N,N));

    delete_vector(&x);
    delete_vector(&v);
    delete_vector(&vTH1);
    delete_vector(&Hv2);
    delete_matrix(&A);
}
```

Phase 2: Eigenvalue revealing iteration scheme

Finding eigenvalues is equivalent to finding the roots of a polynomial (solve characteristic polynomial $= 0$).

Therefore it should not be possible to find eigenvalues exactly in a finite number of steps $\Longrightarrow$ any method must naturally be **iterative** (i.e., Power Iteration, etc . . .)

Then how do we get from a tridiagonal matrix T that has the same eigenvalues as A to a **diagonal** matrix Λ that has the same eigenvalues as A.

Phase 2: Eigenvalue revealing iteration scheme

Idea: repeatedly apply QR decomposition in the following manner:

$A^{(0)} = A$

for $k = 1, 2, 3, \ldots$ **do**

Compute the QR factorization of $A^{(k-1)}$: $Q^{(k)} R^{(k)} = A^{(k-1)}$

Reassamble A by multiplying in reverse order: $A^{(k)} = R^{(k)} Q^{(k)}$

end for

Notes:

- $A^{(k)} = R^{(k)} Q^{(k)}$, but $R^{(k)} = \left[Q^{(k)}\right]^T A^{(k-1)}$, $\Longrightarrow$
  $A^{(k)} = \left[Q^{(k)}\right]^T A^{(k-1)} Q^{(k)}$
- Therefore $A^{(k-1)}$ and $A^{(k)}$ have the same eigenvalues.
- As $k \to \infty$, $A^{(k)} \to \Lambda$

Phase 2: Eigenvalue revealing iteration scheme

$ISUHPC/lectures/lecture15/codes/QRalgorithm/QRA.c:

```
1  #include <stdlib.h>
2  #include <assert.h>
3  #include "trimatrix.h"
4
5  void QRA(trimatrix* T)
6  {
7     // function declarations
8     matrix QRdecomp(matrix* A);
9     int FindZero(trimatrix* T);
10    double GetShift(trimatrix* T);
11
12     // get size
13    const int N = T->rows; assert(N==T->cols);
14
15    if (N==1)
16    { return; }
17    else
18    {
19       int k = FindZero(T); // see if we can deflate
```

Phase 2: Eigenvalue revealing iteration scheme

$ISUHPC/lectures/lecture15/codes/QRalgorithm/QRA.c:

```
        if (k==N) // can't deflate
        {
           // Find shift to improve convergence
           double mu = GetShift(T);

           // Copy tridiagonal matrix into a bigger
           // matrix -- needed for QR step
           matrix R = new_matrix(N,N);
           mget(R,1,1) = tget(T,1,1);
           mget(R,1,2) = tget(T,1,2);
           for (int i=2; i<=(N-1); i++)
              {
                 mget(R,i,i-1) = tget(T,i,i-1);
                 mget(R,i,i)   = tget(T,i,i);
                 mget(R,i,i+1) = tget(T,i,i+1);
              }
           mget(R,N,N-1) = tget(T,N,N-1);
           mget(R,N,N) = tget(T,N,N);

           // Subtract shift from bigger matrix
           for (int i=1; i<=N; i++)
           { mget(R,i,i) -= mu; }
```

Phase 2: Eigenvalue revealing iteration scheme

$ISUHPC/lectures/lecture15/codes/QRalgorithm/QRA.c:

```
            // QR factorization on bigger matrix
            matrix Q = QRdecomp(&R);

            // Multiply Q and R in reverse order
            // and store in a new tridiagonal matrix
            delete_trimatrix(T);
            *T = matrix_mult_to_trimatrix(&R,&Q);

            // Add the shift back in
            for (int i=1; i<=N; i++)
            { tset(T,i,i, tget(T,i,i) + mu ); }

            // Recursive call to QR algorithm
            QRA(T);
        }
        else
        {
            // Split big T matrix into 2 submatrices
            trimatrix T1 = grab_sub_trimatrix(T,1,k);
            trimatrix T2 = grab_sub_trimatrix(T,k+1,N);
```

Phase 2: Eigenvalue revealing iteration scheme

$ISUHPC/lectures/lecture15/codes/QRalgorithm/QRA.c:

```
            // Call QR algorithm on each submatrix
            QRA(&T1);
            QRA(&T2);

            // Merge the result of each submatrix
            // back into larger matrix T
            delete_trimatrix(T);
            *T = merge_sub_trimatrix(&T1,&T2);
            delete_trimatrix(&T1);
            delete_trimatrix(&T2);
        }
    }
}
```

Phase 2: Eigenvalue revealing iteration scheme

$ISUHPC/lectures/lecture15/codes/QRalgorithm/main.c:

```
1  int main()
2  {
3     srand( time(NULL) );
4     int N=0;
5     printf("    Input N: ");
6     scanf("%i",&N);
7     assert(N>1);
8
9     // Create a random symmetric matrix
10    matrix A = new_matrix(N,N);
11
12    for (int i=1; i<=N; i++)
13       for (int j=1; j<=i; j++)
14       {
15          double tmp = ((double)rand())/((double)RAND_MAX);
16          tmp = (double)((int)(10000.0*tmp))/10000.0;
17          mget(A,i,j) = tmp;
18       }
19    for (int i=1; i<=(N-1); i++)
20       for (int j=(i+1); j<=N; j++)
21       { mget(A,i,j) = mget(A,j,i); }
```

Phase 2: Eigenvalue revealing iteration scheme

$ISUHPC/lectures/lecture15/codes/QRalgorithm/main.c:

```
    // Hessenberg reduction to tridiagonal form
    trimatrix T = new_trimatrix(N);
    void Hessenberg(const matrix* A, trimatrix* T);
    Hessenberg(&A,&T);
    printf("\n");
    printf("Original Matrix:\n");
    print_matrix(&A);
    printf("Reduction to Tridiagonal Form:\n");
    print_trimatrix(&T);

    // QR Algorithm to find eigenvalues of T
    // which are the same as the eigenvalues of A
    void QRA(trimatrix* T);
    QRA(&T);
    printf("After QR Algorithm:\n");
    print_trimatrix(&T);
}
```

QR algorithm

```
$ main.exe
    Input N: 5

Original Matrix:

 A =
  |   4.850000e-01,  9.410000e-01,  3.688000e-01,  4.763000e-01,  8.994000e-01 |
  |   9.410000e-01,  5.558000e-01,  7.229000e-01,  3.253000e-01,  7.800000e-01 |
  |   3.688000e-01,  7.229000e-01,  1.958000e-01,  9.785000e-01,  7.581000e-01 |
  |   4.763000e-01,  3.253000e-01,  9.785000e-01,  8.200000e-02,  6.706000e-01 |
  |   8.994000e-01,  7.800000e-01,  7.581000e-01,  6.706000e-01,  9.592000e-01 |

Reduction to Tridiagonal Form:

 T =
  |   4.850000e-01, -1.434321e+00,  0.000000e+00,  0.000000e+00,  0.000000e+00 |
  |  -1.434321e+00,  2.356596e+00,  8.339285e-01,  0.000000e+00,  0.000000e+00 |
  |   0.000000e+00,  8.339285e-01, -5.327207e-02,  5.146370e-01,  0.000000e+00 |
  |   0.000000e+00,  0.000000e+00,  5.146370e-01, -4.834033e-01,  4.511134e-03 |
  |   0.000000e+00,  0.000000e+00,  0.000000e+00,  4.511134e-03, -2.712059e-02 |

After QR Algorithm:

 T =
  |   3.299423e+00,  0.000000e+00,  0.000000e+00,  0.000000e+00,  0.000000e+00 |
  |   0.000000e+00, -9.664793e-01,  0.000000e+00,  0.000000e+00,  0.000000e+00 |
  |   0.000000e+00,  0.000000e+00,  3.322394e-01,  0.000000e+00,  0.000000e+00 |
  |   0.000000e+00,  0.000000e+00,  0.000000e+00, -3.602803e-01,  0.000000e+00 |
  |   0.000000e+00,  0.000000e+00,  0.000000e+00,  0.000000e+00, -2.710241e-02 |
```

More on code optimization

Sample compiler optimization

See

https://gcc.gnu.org/onlinedocs/gcc-11.2.0/gcc/Optimize-Options.html#Optimize-Opt

for a list of many gcc optimization flags.

Often `-O2` or `-O3` is used to include many common optimizations.

Common subexpression elimination

`-fgcse (or -02, -03) optimization flag` will replace:

```
for (int i=0; i<N; i++)
{
    y[i] = 2.0*x[i]*pi;
}
```

by the machine code equivalent of ...

```
pi2 = 2.0*pi;
for (int i=0; i<N; i++)
{
    y[i] = pi2*x[i];
}
```

Note: this may give slightly different results because computer arithmetic is non-commutative!

Function inlining

`-finline-functions (or -03) optimization flag` will replace function calls by the corresponding code inline:

For example if the following functions are defined somewhere:

```
double  f(double x){ return (x*x - 4.0); }
double fp(double x){ return (2.0*x);     }
```

and then called as follows:

```
// evaluate function and its derivative
double fx = f(x);
double fxprime = fp(x);
```

then the compiler will replace by the machine code equivalent of ...

```
fx = x*x - 4.0;
fxprime = 2.0*x;
```

Note: Overhead of function call is avoided. Can make a big difference if $f(x)$ is repeatedly evaluated inside of a loop.